Cheat Sheet — Parallel Algorithms with CUDA (Final)

PRAM · Work-Step · Reduce/Scan/Histogram · CUDA implementation & fine-tuning

1. PRAM Model

PRAM = p processors (id $1..p$) + **shared memory** หนึ่งชุด, รันแบบ synchronous/SIMD lock-step, 1 คำสั่ง = 1 หน่วยเวลา, **ไม่คิดต้นทุน** sync/communication ⇒ เป็น baseline ของ concurrency ("ถ้าดีบน PRAM ไม่ได้ ก็ดีบนเครื่องจริงไม่ได้")

EREW	Exclusive Read, Exclusive Write (อ่อนที่สุด, implement ง่ายสุด)
CREW	อ่านพร้อมกันได้ เขียนพร้อมกันไม่ได้
CRCW	Common (ทุกตัวต้องเขียนค่าเดียวกัน) / Arbitrary (ใครก็ได้ชนะ) / Priority (id น้อยสุดชนะ) / Combining (รวมค่า เช่น sum)

ความแรง: EREW ≤ CREW ≤ Common ≤ Arbitrary ≤ Priority ≤ Combining. EREW simulate Priority CRCW ได้ช้าลง $O(\log p)$. Broadcast บน EREW = $\Theta(\log p)$ (doubling copy).

นิยามตัววัด ($T^*(n)$ = เวลา optimal/best sequential)

$S_p = \frac{T^*}{T_p}$ (speedup), $C_p = pT_p$ (cost), $E_p = \frac{S_p}{p} = \frac{T^*}{pT_p}$

Optimal speedup: $S_p = \Theta(p) \iff E_p = \Theta(1) \iff$ **cost-optimal** $C_p = \Theta(T^*)$

Speedup เกิน $O(p)$ ไม่ได้: ถ้า $S_p = \omega(p)$ ให้ 1 processor simulate p ตัวทีละ step ได้เวลา $pT_p = o(T^*)$ ขัดกับความ optimal ของ T^* (superlinear จริงมาจาก cache เท่านั้น)

ตัวอย่างมาตรฐาน: Sum / Product / Max ของ n ค่า (Quiz 2)

1. แบ่ง p block ขนาด n/p, แต่ละ processor รวมแบบ sequential: $O(n/p)$
2. รวม p ค่าด้วย balanced binary tree: $O(\log p)$

$T_p = O(n/p + \log p)$, $C_p = O(n + p\log p)$ ⇒ optimal เมื่อ $p\log p = O(n)$ คือ $p = O(n/\log n)$; เลือก $p = n/\log n$ ได้ $T_p = O(\log n)$, $C_p = O(n)$.

กับดัก: $p = n$ ทำ tree ตรง ๆ ได้ $T = \log n$ แต่ $C = n\log n$ ⇒ **ไม่** cost-optimal (speedup $n/\log n \neq \Theta(p)$)

ตัวเลข ($T_p = (n/p - 1) + \log_2 p$, $n = 1024$, $T^* = 1023$):

p	local	tree	T_p	S_p	E_p
32	31	5	36	28.4	0.89
128	7	7	14	73.1	0.57
1024	0	10	10	102.3	0.10

p เกิน $n/\log n \approx 102$ แล้ว $\log p$ ครอบงำ → efficiency ตก (ตรงกับเงื่อนไข $p = O(n/\log n)$)

วิเคราะห์ cost แบบทั่วไป: ถ้า $T_p = O(\frac{n\log n}{p} + \log^2 n)$, $T^* = n\log n$: $C_p = O(n\log n + p\log^2 n)$ ⇒ optimal เมื่อ $p = O(n/\log n)$; $p = n$ ได้ speedup $n/\log n = o(p)$ **ไม่ optimal**

ข้อจำกัด PRAM & ความเชื่อมโยงกับ GPU

PRAM สมมุติ memory latency คงที่ (ไม่จริงเมื่อ p ใหญ่). แต่ถ้าให้ processor จริง 1 ตัว simulate PRAM processor หลายตัว จะ **ซ่อน latency** ได้ = แนวคิด GPU (thread ≫ core, warp scheduler สลับ warp ที่รอ memory).

CRCW tricks

- **Max ใน $O(1)$** (Common CRCW, n^2 proc): **forall** i: $m_i \leftarrow 1$; **forall** (i,j): if $a_i < a_j$ then $m_i \leftarrow 0$; **forall** i: if $m_i = 1$ then $max \leftarrow a_i$. $S = O(1)$, $W = O(n^2)$ **ไม่ work-optimal**. (Cascade/doubly-log tree ได้ $S = O(\log\log n)$, $W = O(n)$)
- **OR ของ n bit ใน $O(1)$** (Common): $r \leftarrow 0$; forall i: if b_i then $r \leftarrow 1$. $W = O(n)$.
- Combining CRCW = มี fetch-and-add ≈ `atomicAdd` ใน CUDA (แต่จริงถูก serialize)

2. Work-Step (Work-Time / Work-Depth) Model

ซ่อนจำนวน processor: แต่ละ step ทำ operation พร้อมกันได้ **ไม่จำกัด** ผ่าน `for all ... in parallel`; ใน forall หนึ่งครั้ง RHS ทุกตัวถูก evaluate **ก่อน** เขียน LHS (synchronous).

Work / Step / Brent

Work $W(n) = \sum_{i=1}^{S} W_i$ = จำนวน op ทั้งหมด (= เวลาบน 1 proc)

Step $S(n)$ = จำนวน parallel step (= เวลาเมื่อ proc ไม่จำกัด = critical path)

Parallelism = W/S. ต่อกัน: W บวก, S บวก. ขนานกัน: W บวก, S = **max**

Brent: $\max\left(\frac{W}{p}, S\right) \le T_p \le \left\lfloor \frac{W}{p} \right\rfloor + S$

พิสูจน์: step i มี W_i op, ใช้ p proc ได้ $\lceil W_i/p \rceil \le \lfloor W_i/p \rfloor + 1$; รวมทุก step $\le \lfloor W/p \rfloor + S$

Work-optimal & Minimum step

Work-optimal (work-efficient): $W(n) = \Theta(T^*(n))$

ทำไมสำคัญที่สุด: ถ้า $W = \omega(T^*)$: $S_p = \frac{T^*}{W/p + S} \le p\frac{T^*}{W} \xrightarrow{n\to\infty} 0$ — p จริงมีจำกัด แต่ n โตได้เรื่อย ๆ ⇒ เสีย speedup ไม่มีขอบเขต (+เปลือง energy)

$W = o(T^*)$ **เป็นไปไม่ได้**: รันบน $p = 1$ ได้เวลา $O(W)$ เร็วกว่า optimal sequential — ขัดแย้ง

เมื่อ work-optimal: $T_p = O(T^*/p + S)$ ⇒ optimal speedup ตราบที่ $p = O(T^*/S)$

⇒ ในกลุ่ม work-optimal ตัวที่ **step น้อยกว่า scalable กว่า** (ใช้ proc ได้มากกว่า)

Lower bound ของ step: output ขึ้นกับ input ทั้ง n ตัว + operator fan-in 2 (EREW/CREW) แต่ละ step รวมข้อมูลได้แค่ 2 เท่า ⇒ $S = \Omega(\log n)$ (เช่น reduce, scan)

เปรียบเทียบ algorithm (สไตล์ Quiz 2)

ลำดับ: **(1) work-optimal ก่อน (2) แล้ว step น้อย (3) ค่าคงที่ไม่นับ**

- A: $W = T^*$, $S = \sqrt{n}$ vs B: $W = T^*\log n$, $S = \log n$ ⇒ **A ดีกว่า**: $T_p^A = O(T^*/p + \sqrt{n})$ optimal ทุก $p = O(T^*/\sqrt{n})$; $T_p^B \ge T^*\log n/p$ ได้ speedup $\le p/\log n$ ไม่ optimal และแย่ลงเมื่อ n โต
- A: $W = T^*\log n$, $S = \log n$ vs B: $W = T^*$, $S = 2\log n$ ⇒ **B ดีกว่า** (work-optimal, step asymptotic เท่ากัน)
- A: $W = T^*$, $S = \log^2 n$ vs B: $W = T^*$, $S = \log n$ ⇒ **B** (ทั้งคู่ optimal แต่ B ใช้ p ได้ถึง $T^*/\log n$)

ลักษณะ parallel solution ที่ดี

(1) work-optimal (2) step polylog (parallelism W/S สูง) (3) ใช้ memory model อ่อนสุด (EREW ≻ CRCW) (4) pseudocode ขัด map ลง hardware ได้

เขียน WS pseudocode: reduce (balanced tree), $n = 2^k$

```
for all 1 ≤ i ≤ n in parallel do
    B[i] ← A[i]
end for
for h = 1 to log n do                          ▷ sequential loop ของระดับ
    for all 1 ≤ i ≤ n/2^h in parallel do
        B[i] ← B[2i − 1] ⊕ B[2i]
    end for
end for
return B[1]
```

$S = \log n + 2 = O(\log n)$; $W = n + \sum_h n/2^h + 1 = O(n)$ ⇒ **work-optimal, EREW**

Cascaded sum (algorithm cascading) — work-optimal และ $S = O(\log n)$

```
k ← log n; m ← n/k                             ▷ แบ่ง m block ขนาด k
```

for all $1 \le i \le m$ **in parallel do**
  $B[i] \leftarrow A[(i-1)k+1]$
  for $j = 2$ **to** k **do** $B[i] \leftarrow B[i] + A[(i-1)k+j]$
  end for
end for
$s \leftarrow \text{TreeReduce}(B[1..m])$

Phase 1: $W = O(n)$, $S = O(\log n)$; Phase 2: $W = O(n/\log n)$, $S = O(\log n) \Rightarrow W = O(n)$, $S = O(\log n)$; Brent: $T_p = O(n/p + \log n)$ = ตัวเดียวกับ PRAM ข้างบน. **= thread coarsening ใน CUDA**

Pointer jumping (list ranking / หา root ของ forest)

for all i **in parallel do**
  $r[i] \leftarrow (next[i] = \text{nil})\ ?\ 0 : 1$
end for
for $t = 1$ **to** $\lceil \log n \rceil$ **do**
  for all i with $next[i] \neq \text{nil}$ **in parallel do**
    $r[i] \leftarrow r[i] + r[next[i]]$; $next[i] \leftarrow next[next[i]]$
  end for
end for

$S = O(\log n)$, $W = O(n \log n)$ ไม่ **work-optimal** ($T^* = O(n)$); CREW (หลายตัวอ่าน node ท้ายพร้อมกัน)

Recurrence ที่ใช้บ่อย

Recurrence	ผล	ใช้กับ
$S(n) = S(n/2) + O(1)$	$O(\log n)$	step ของ reduce/scan
$W(n) = W(n/2) + O(n)$	$O(n)$	work ของ recursive scan
$W(n) = 2W(n/2) + O(1)$	$O(n)$	D&C reduce
$S(n) = S(n/2) + O(\log n)$	$O(\log^2 n)$	merge sort (parallel merge)
$W(n) = 2W(n/2) + O(n)$	$O(n \log n)$	merge sort
$S(n) = S(\sqrt{n}) + O(1)$	$O(\log \log n)$	doubly-log tree

เทคนิคการออกแบบ

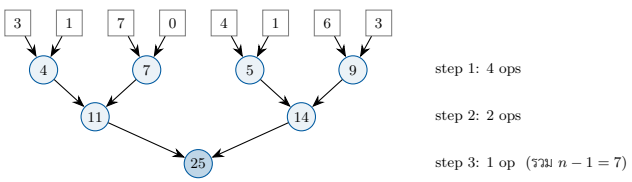
Balanced tree sweep ขึ้น/ลงบน tree สมมุติ: reduce, scan, broadcast
Pointer jumping doubling ตาม pointer: list ranking, root finding ($W = n \log n$)
Algorithm cascading sequential บน block ขนาด $\log n$ + parallel tree บน $n/\log n$ ค่า $\Rightarrow$ ลด work เป็น $O(n)$
Divide & conquer แบ่ง, แก้ขนาน, combine: merge sort, quicksort, convex hull

ตัวอย่าง WS อื่น ๆ (W / S)

ปัญหา	W	S
Inner product $\sum a_i b_i$: map $\times$ $\rightarrow$ reduce	$O(n)$	$O(\log n)$
Matrix–vector $y = Ax$ ($n \times n$): forall $(i,j) \rightarrow$ reduce แต่ละแถว	$O(n^2)$	$O(\log n)$
Matrix multiply $n \times n$ (forall i,j,k + reduce)	$O(n^3)$	$O(\log n)$
Merge sort (merge แบบ parallel $O(\log n)$)	$O(n \log n)$	$O(\log^2 n)$
Bitonic sort	$O(n \log^2 n)$	$O(\log^2 n)$

3. Reduce

Input $\langle a_0..a_{n-1}\rangle$ + **associative** $\oplus$ ($+, \times, \max, \min$, AND, OR, ...). Output $a_0 \oplus \cdots \oplus a_{n-1}$. $T^* = n - 1$; tree: $n-1$ op ใน $\lceil \log n \rceil$ step. ต้องการ associativity เท่านั้น (ถ้า implementation สลับลำดับ เช่น grid-stride ต้อง commutative ด้วย). **float + ไม่ associative จริง** $\rightarrow$ ผลต่างจาก sequential เล็กน้อย (round-off).



4. Scan (Prefix sum)

Inclusive: $\langle a_0, a_0 \oplus a_1, \ldots\rangle$; **Exclusive**: $\langle I, a_0, a_0 \oplus a_1, \ldots, a_0 \oplus .. \oplus a_{n-2}\rangle$
$\text{incl}[i] = \text{excl}[i] \oplus a_i$; $\text{excl}[i] = \text{incl}[i-1]$ (shift ขวาใส่ identity I)
ตัวอย่าง $\langle 3,1,7,0,4,1,6,3\rangle$: incl $= \langle 3,4,11,11,15,16,22,25\rangle$, excl $= \langle 0,3,4,11,11,15,16,22\rangle$

Hillis–Steele (inclusive) — step-efficient, ไม่ work-efficient

for $d = 1$ **to** $\log n$ **do**
  for all $0 \le k < n$ **in parallel do**
    if $k \ge 2^{d-1}$ **then** $x[k] \leftarrow x[k - 2^{d-1}] \oplus x[k]$
    end if
  end for
end for

$S = \log n$; $W = \sum_d (n - 2^{d-1}) = n \log n - (n-1) = O(n \log n)$. ใน CUDA ต้อง **double buffer** (forall อ่านค่าเก่าก่อนเขียน). เหมาะกับ n เล็ก (scan ใน warp/block).
Trace $\langle 3,1,7,0,4,1,6,3\rangle$: d=1: 3, 4, 8, 7, 4, 5, 7, 9; d=2: 3, 4, 11, 11, 12, 12, 11, 14; d=3: 3, 4, 11, 11, 15, 16, 22, 25

Blelloch (exclusive) — work-efficient: up-sweep + down-sweep

for $d = 0$ **to** $\log n - 1$ **do** ▷ up-sweep (= reduce)
  for all $k = 0$ **to** $n-1$ **by** 2^{d+1} **in parallel do**
    $x[k + 2^{d+1} - 1] \leftarrow x[k + 2^d - 1] \oplus x[k + 2^{d+1} - 1]$
  end for
end for
$x[n-1] \leftarrow I$
for $d = \log n - 1$ **downto** 0 **do** ▷ down-sweep
  for all $k = 0$ **to** $n-1$ **by** 2^{d+1} **in parallel do**
    $t \leftarrow x[k + 2^d - 1]$; $x[k + 2^d - 1] \leftarrow x[k + 2^{d+1} - 1]$
    $x[k + 2^{d+1} - 1] \leftarrow t \oplus x[k + 2^{d+1} - 1]$
  end for
end for

เริ่ม	3, 1, 7, 0, 4, 1, 6, 3
up d=0	3, **4**, 7, **7**, 4, **5**, 6, **9**
up d=1	3, 4, 7, **11**, 4, 5, 6, **14**
up d=2	3, 4, 7, 11, 4, 5, 6, **25** (root = total)
root$\leftarrow$ 0	3, 4, 7, 11, 4, 5, 6, **0**
down d=2	3, 4, 7, **0**, 4, 5, 6, **11**
down d=1	3, **0**, 7, **4**, 4, **11**, 6, **16**
down d=0	0, 3, 4, 11, 11, 15, 16, 22

แต่ละเฟส $n-1$ op, $\log n$ step $\Rightarrow W \approx 2(n-1) = O(n)$, $S = 2\log n$, EREW. Invariant down-sweep: ค่าที่ส่งเข้า node = "ผลรวมของทุกตัวทางซ้ายของ subtree". ใช้กับ operator **non-commutative** ได้ (รักษาซ้าย/ขวา: down-sweep ทำ $t \oplus x$ คือ ซ้าย$\oplus$ขวา).

Recursive scan (pram.pdf Alg. 4)

$y_i = x_{2i-1} \oplus x_{2i}$ ($n/2$ ค่า) $\rightarrow$ scan y แบบ recursive ได้ $z \rightarrow s_{2i} = z_i$, $s_1 = x_1$, $s_{2i+1} = z_i \oplus x_{2i+1}$. $S(n) = S(n/2) + O(1) = O(\log n)$, $W(n) = W(n/2) + O(n) = O(n)$, EREW.

Primitives จาก scan (ทุกตัว $W = O(n)$, $S = O(\log n)$)

Enumerate	flag→0/1 → exclusive +scan = ลำดับที่ของตัวที่ true
Copy/Broadcast	$\langle v, I, I, ..\rangle$ → scan
Pack/Filter/Compact	flag → excl. scan = index ปลายทาง → scatter ตัวที่ true; ขนาด = scan สุดท้าย+flag สุดท้าย
Split (stable)	i_{down}=enumerate($\neg F$); $i_{up} = n-$ (reverse-scan ของ F) หรือ $\#false$+enumerate(F); ย้าย false ไปหน้า true ไปหลัง
Radix sort	split ทีละ bit ตั้งแต่ LSB (b bit: $W = O(bn)$, $S = O(b \log n)$)
Segmented scan	scan บนคู่ (flag, value): restart ที่หัว segment
Flatten / allocate	scan ขนาด sub-array ได้ offset แล้ว copy ขนาน

Split ตัวอย่าง $A = \langle 5,2,7,4,1,6\rangle$ bit 0: $F = \langle 1,0,1,0,1,0\rangle$ enumerate($\neg F$)=$\langle \cdot,0,\cdot,1,\cdot,2\rangle$, #false= 3, enumerate($F$) +3=$\langle 3,\cdot,4,\cdot,5,\cdot\rangle$ ⇒ $\langle 2,4,6,5,7,1\rangle$; bit 1 ⇒ $\langle 4,5,1,2,6,7\rangle$; bit 2 ⇒ $\langle 1,2,4,5,6,7\rangle$ **sorted**

5. Histogram

Input $A[1..n]$ ค่าใน $[0,m)$ (หรือ map ด้วย f เข้า bin). Output $H[j] = |\{i : A[i] = j\}|$. Sequential: `for i: H[A[i]]++` ⇒ $T^* = O(n+m)$. **ความยาก**: output conflict — หลาย thread เขียน bin เดียวกัน (ตำแหน่ง output ไม่รู้ล่วงหน้า ต่างจาก reduce/scan)

วิธี	W	S	หมายเหตุ
Matrix: $R[i][j] = [A_i = j]$ → reduce แต่ละ column	$O(nm)$	$O(\log n)$	ไม่ optimal
Cascaded (Alg. 6): n/m block ขนาด m, sequential private hist ต่อ block, แล้ว reduce ต่อ bin	$O(n)$	$O(m+\log n)$	**optimal**, EREW
CRCW/ fetch-and-add (`atomicAdd`)	$O(n)$	$O(1)^*$	*จริงถูก serialize

Privatization

แต่ละกลุ่มงาน (block) มี **private histogram** → **merge** ทีหลัง: เปลี่ยน conflict แบบ fine-grained (n ครั้ง) เป็น reduce แบบ coarse (#block × m ครั้ง) — รากฐานของทั้ง Alg. 6 และ CUDA shared-memory histogram

Counting sort = histogram → exclusive scan (ตำแหน่งเริ่มของแต่ละ key) → scatter; stable ถ้าทำ histogram ต่อ block แล้ว scan แบบ column-major (bin ก่อน block) ตาม Fig. 13 ใน parallel.pdf. $W = O(n+m)$.

Median/percentile (int) = histogram → inclusive scan → bin แรกที่ prefix $\geq \lceil n/2 \rceil$ (map+reduce min)

6. Decompose ปัญหาใหม่ → building blocks

สูตรคิด (เขียนลงข้อสอบเป็นขั้น ๆ)

1. **Map** (แต่ละตัวอิสระ): flag/คำนวณย่อย — $W = O(n)$, $S = O(1)$
2. ต้องการ **คำตอบเดียว** → **Reduce**
3. ต้องการ **คำตอบต่อตำแหน่งที่ขึ้นกับตัวก่อนหน้า** (running, ตำแหน่งปลายทาง) → **Scan**
4. ต้องการ **นับ/รวมตามหมวด** (key→bin) → **Histogram**
5. ตรวจ **associative** + หา **identity**; ถ้าไม่มีให้นิยาม operator บน tuple
6. รวมค่า: ต่อกันทุกขั้น $O(n)/O(\log n)$ ⇒ ทั้งหมด $W = O(n)$, $S = O(\log n)$ **work-optimal**

ปัญหา	Decomposition
นับจำนวน $> t$	map $[a_i > t]$ → reduce(+)
Mean / variance	reduce(+) บน a_i และ a_i^2; $\sigma^2 = \overline{x^2} - \bar{x}^2$ (หรือ tuple $(n, \Sigma x, \Sigma x^2)$)
Index แรกที่ $> t$	map i ถ้า $a_i > t$ ไม่งั้น ∞ → reduce(min)
argmax	reduce บนคู่ (a_i, i): เลือกค่ามากกว่า, เสมอเลือก i น้อย
Compact (ลบ 0)	map flag → excl. scan(+) → scatter
Running max	inclusive scan(max)
Line of sight	map มุม $\theta_i = (h_i - h_0)/i$ → excl. scan(max) → map: เห็นถ้า $\theta_i > \max_{j<i} \theta_j$
วงเล็บสมดุล	map (→ +1,) → −1 → incl. scan(+) → reduce(min)≥ 0 และตัวสุดท้าย = 0
Polynomial $\sum a_i x^i$	map ทุกตัว = x → excl. scan(×,1) ได้ x^i → map $a_i x^i$ → reduce(+)
$x_i = a_i x_{i-1} + b_i$	scan บนคู่ (a,b): $(a_1,b_1) \otimes (a_2,b_2) = (a_1a_2,\ a_2b_1 + b_2)$, identity $(1,0)$ — associative แต่ **ไม่ commutative**; $x_i = b$ ของผล (เมื่อ $x_0 = 0$)
Counting sort	histogram → excl. scan → scatter
Run-length encoding	map head flag $[a_i \neq a_{i-1}]$ → excl. scan = index run → scatter ค่า+ตำแหน่งเริ่ม → map length = $\text{start}_{k+1} - \text{start}_k$
Lexical/DFA	map สัญลักษณ์ → transition function → scan ด้วย composition
Sort (key เล็ก)	radix sort = split ทีละ bit / counting sort ทีละ digit
Top-k / threshold	histogram → scan จากท้าย หา bin ที่สะสมถึง k → filter

Line of sight ตัวอย่าง $h = \langle 10,12,11,15,14,22,26\rangle$ ($h_0 = 10$ ที่จุดสังเกต): $\theta_{1..6} = \langle 2, 0.5, 1.67, 1, 2.4, 2.67\rangle$; excl max = $\langle -\infty, 2, 2, 2, 2, 2.4\rangle$ ⇒ เห็นจุด **1, 5, 6**.

Polynomial $1 + 2x + 3x^2$ ที่ $x = 2$: $x^i = \langle 1,2,4\rangle$, $a_i x^i = \langle 1,4,12\rangle$, sum = 17.

7. Filter / Flatten / Search / Merge / K-th smallest

Filter(A, p) — stable

forall $F_i \leftarrow [p(A_i)] \rightarrow (X, m) \leftarrow$ excl. scan(F) → forall: if F_i then $R[X_i] \leftarrow A_i$. $W = O(n)$, $S = O(\log n)$, EREW. $A = \langle 4,9,1,7,3,8,2,6\rangle$, $x > 5$: $F = \langle 0,1,0,1,0,1,0,1\rangle$, $X = \langle 0,0,1,1,2,2,3,3\rangle$, $m = 4$, $R = \langle 9,7,8,6\rangle$. CUDA: flag ใน register + ballot/popc + scan ของ block counts; `atomicAdd` จองที่ = ไม่ stable

Flatten(A) (nested → flat)

forall $\text{sizes}_i \leftarrow |A_i| \rightarrow (X, m) \leftarrow$ excl. scan(sizes) → forall i, forall j: $R[X_i + j] \leftarrow A_i[j]$. $W = O(n+m)$, $S = O(\log n)$. $\langle\langle 3,1,2\rangle, \langle 5,1\rangle, \langle\rangle, \langle 7,8,4,9\rangle\rangle$: sizes $3,2,0,4$, $X = 0,3,5,5$, $m = 9$. CUDA: 1 thread/sub-array → **load imbalance**; ใช้ 1 thread/output j: $i = \texttt{upper_bound}(X, j) - 1$, $R[j] = A_i[j - X_i]$ (coalesced, งานเท่ากัน)

Search (sorted, หา $\max\{i : A_i < v\}$)

Binary: $W = S = O(\log n)$. **k-way**: เทียบ v กับขอบ k block พร้อมกัน, ลดขนาด k เท่า/รอบ ⇒ $\log_k n$ รอบ, $W = O(k \log_k n)$; $k = \sqrt{n}$ ⇒ $S = O(1)$, $W = O(\sqrt{n})$ ไม่ work-optimal, CREW. $A = 2,4,..,32$, $v = 19$, $k = 4$: เทียบ $2, 10, 18, 26$ → block $A[8..11]$ → $18 < 19 < 20$ → ตำแหน่ง 8. q keys: binary search พร้อมกัน $W = O(q \log n)$, $S = O(\log n)$

kth / co-rank ของ 2 sorted arrays

หา $i = l_a$ ($l_b = k - i$) ที่ $A[i-1] \leq B[k-i]$ **และ** $B[k-i-1] < A[i]$; binary search $O(\log \min(k, |A|))$. $A = \langle 1,4,6,9\rangle$, $B = \langle 2,3,7,8\rangle$: $k = 4 \rightarrow (2,2)$, $k = 5 \rightarrow (3,2)$

Merge ($m = |A| + |B|$)

วิธี	W	S
Rank: $\text{pos}(A_i) = i + \#\{B < A_i\}$, $\text{pos}(B_j) = j + \#\{A \le B_j\}$	$O(m \log m)$ **N**	$O(\log m)$
D&C: kth ครึ่ง output แล้ว recurse 2 ด้าน	$O(m)$	$O(\log^2 m)$
Merge path: p co-rank + sequential merge, $p = m/\log m$	$O(m)$	$O(\log m)$

$W(m) = 2W(\frac{m}{2}) + O(\log m) = O(m)$, $S(m) = S(\frac{m}{2}) + O(\log m) = O(\log^2 m)$; merge path: $W = O(m + p \log m)$, $S = O(\frac{m}{p} + \log m)$. Rank ตัวอย่าง: $A \to 0, 3, 4, 7$; $B \to 1, 2, 5, 6 \Rightarrow \langle 1, 2, 3, 4, 6, 7, 8, 9\rangle$ ($<$ กับ $\le$ ทำให้ stable ไม่ชน). CUDA: thread ละ T output (load balance), block โหลด tile เข้า smem แบบ coalesced แล้วเขียนผ่าน smem

K-th smallest (quickselect)

$p \leftarrow$pivot; $L \leftarrow$filter($< p$), $G \leftarrow$filter($> p$); if $k < |L|$: recurse L; elif $k \ge |A| - |G|$: recurse G ด้วย $k - (|A| - |G|)$; else ตอบ p. Random pivot: $W = O(n)$ **exp.**, $S = O(\log^2 n)$ **whp**; worst $W = O(n^2)$; median-of-medians $W = O(n)$ แต่ $S = O(n^{0.84})$; sort ก่อน $W = O(n \log n)$. $\langle 7, 2, 9, 4, 3, 8, 1, 6\rangle$, $k = 3$, pivot ตัวแรก: p=7 L=2, 4, 3, 1, 6 $\to p$=2, G=4, 3, 6, k=1 $\to p$=4 **ตอบ 4**. **Radix select (GPU)**: histogram 8 bit บน $\to$ scan $\to$ หา bucket ที่มีตำแหน่ง $k \to$ filter $\to$ ซ้ำ (≤ 4 รอบ); traffic $\approx 2n$ (รอบแรก) เทียบ radix sort $\approx 12n$

8. CUDA: execution & memory model

- grid $\to$ block $\to$ thread; **warp = 32 thread** รัน SIMT; block อยู่บน **SM** เดียวตลอดชีวิต
- `__syncthreads()` = barrier **ภายใน block เท่านั้น** (ห้ามอยู่ใน branch ที่ thread ไปไม่ครบ); **global barrier** = จบ kernel แล้ว launch ใหม่ (หรือ cooperative groups grid sync)
- Index: `i = blockIdx.x*blockDim.x + threadIdx.x;` grid-stride: `i += blockDim.x*gridDim.x`
- Warp divergence: branch ที่ thread ใน warp เดียวกันไปคนละทาง $\to$ ทำทั้งสองทางแบบ serialize

Memory	Scope	Speed	หมายเหตุ
Register	thread	เร็วสุด	จำกัด/ SM $\to$ occupancy; spill ไป local (ช้า)
Shared	block	on-chip	32 banks $\times$ 4 B; bank conflict
L1/L2	SM/GPU	กลาง	cache line 128 B, sector 32 B
Global	grid	ช้า (400–800 cyc)	ต้อง coalesced
Constant	grid RO	เร็วเมื่อ broadcast	ทุก thread อ่าน ค่าเดียวกัน

WS / PRAM	CUDA
`for all` i `in parallel`	1 thread ต่อ i (หรือ grid-stride)
1 parallel step	`__syncthreads()` (ใน block) / kernel launch ใหม่ (ทั้ง grid)
Synchronous semantics	double buffer หรือ อ่านลง register $\to$ sync $\to$ เขียน $\to$ sync
p processors (Brent)	thread ที่ resident จริง $\approx$ #SM $\times$ thread/SM
Algorithm cascading	**thread coarsening**: thread ละหลาย element แบบ sequential แล้วค่อย tree
Privatization	histogram ใน shared memory ต่อ block + merge ด้วย `atomicAdd`
Concurrent write (CRCW)	atomics (serialize เมื่อชน)
Hierarchy	warp (shuffle) $\to$ block (shared) $\to$ grid (global/launch)

Coalescing (warp 32 thread, `float` 4 B, sector 32 B)

Pattern	Sectors	Efficiency
`a[i]` ติดกัน, aligned	4 (128 B)	100%
`a[i+1]` misaligned	5	80%
`a[2*i]` stride 2	8	50%
`a[8*i]` stride ≥ 8	32	12.5%
AoS `struct{x,y,z}` อ่านแค่ `x`		$\approx$33% $\to$ ใช้ SoA
ทุก thread อ่าน address เดียว	1	broadcast

Shared memory bank conflict (32 banks, bank = (addr/4) mod 32)

Pattern (thread t ใน warp)	ผล
`s[t]`	ไม่มี conflict
`s[2*t]`	2-way
`s[32*t]` (เช่น column ของ `s[32][32]`)	32-way $\to$ padding `s[32][33]`
`s[0]` ทุกตัว (address เดียวกัน)	ไม่ conflict (broadcast)
stride คี่ใด ๆ (`s[3*t]`)	ไม่มี conflict

Occupancy

$$\text{blocks/SM} = \min\left(\left\lfloor \frac{\text{maxThr/SM}}{\text{thr/blk}} \right\rfloor, \left\lfloor \frac{\text{regs/SM}}{\text{reg/thr}\cdot\text{thr/blk}} \right\rfloor, \left\lfloor \frac{\text{smem/SM}}{\text{smem/blk}} \right\rfloor, \text{maxBlk/SM}\right)$$

$$\text{occupancy} = \frac{\text{blocks/SM} \times \text{thr/blk}}{\text{maxThr/SM}}$$ (คิดเป็น warp ก็ได้).

ตัวอย่าง (2048 thr/SM, 64K reg, 64 KB smem, 32 blk/SM):

Config	ตัวจำกัด	Occ.
256 thr, 32 reg, 0 smem	threads (8 blk)	100%
256 thr, 64 reg	regs: $65536/(64 \cdot 256) = 4$ blk	50%
256 thr, 32 reg, 32 KB smem	smem: 2 blk	25%
32 thr/blk	max 32 blk $\to$ 1024 thr	50%

Occupancy สูงช่วย **latency hiding** แต่ไม่จำเป็นต้อง 100% — ตัดสินจาก GB/s ที่วัดได้จริง

9. Reduce บน CUDA

4.1 Global memory อย่างเดียว: 1 tree level = 1 kernel launch

```
__global__ void reduceStep(float *x, int len, int half) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < len / 2) x[i] += x[i + half];   // sequential addressing,
      coalesced
}
for (int len = n; len > 1; len = (len + 1) / 2) {      // host:
     ceil(log n) launches
  int half = (len + 1) / 2, T = 256, B = (len / 2 + T - 1) / T;
  reduceStep<<<B, T>>>(d_x, len, half);
}                                                  // ans = d_x[0]
```

ข้อเสีย: traffic $\approx 3n/2 + 3n/4 + \cdots \approx 3n$ element, $\log n$ launch overhead, ไม่ reuse.

4.2 Per-block shared memory (partial sums)

```
__global__ void reduceShared(const float *in, float *out, int n) {
  extern __shared__ float s[];
  unsigned t = threadIdx.x, i = blockIdx.x * (blockDim.x * 2) + t;
  float v = 0;                                     // first add
     during load
  if (i < n) v = in[i];
  if (i + blockDim.x < n) v += in[i + blockDim.x];
  s[t] = v; __syncthreads();
  for (unsigned st = blockDim.x / 2; st > 0; st >>= 1) {
    if (t < st) s[t] += s[t + st];                 // no divergence
     in full warps,
    __syncthreads();                               // no bank
     conflict
  }
  if (t == 0) out[blockIdx.x] = s[0];              // then reduce out
     [] again / atomicAdd
}
```

Optimization ladder (Harris, G80, 2^{22} int)

Version	ปัญหา → principle	ms
1 Interleaved `tid%(2s)==0`	divergence สูง + `%` ช้า	8.05
2 `idx=2*s*tid`	แก้ divergence แต่ **bank conflict**	3.46
3 Sequential addressing	ไม่มี conflict, active thread ติดกัน	1.72
4 First add during load	ครึ่ง thread ว่างตั้งแต่ step แรก	0.97
5 Unroll last warp	ไม่ต้อง sync ใน warp สุดท้าย	0.54
6 Complete unroll (template)	ลด loop/instruction overhead	0.38
7 Multiple elem/thread	**cascading/Brent**, ลด block, memory-bound	0.27

Kernel 7 ≈ 62.7 GB/s จาก peak 86.4 GB/s (≈73%), เร็วขึ้น ≈30×

Final: grid-stride + warp shuffle + 1 atomic/block

```
__inline__ __device__ float warpSum(float v) {
  for (int off = 16; off > 0; off >>= 1)
    v += __shfl_down_sync(0xffffffff, v, off);   // register-to-
     register, no smem
  return v;
}
__global__ void reduceFinal(const float *in, float *out, int n) {
  __shared__ float ws[32];
  float v = 0;
  for (int i = blockIdx.x * blockDim.x + threadIdx.x; i < n;
       i += blockDim.x * gridDim.x) v += in[i];  // coalesced +
     coarsened
  v = warpSum(v);
  int lane = threadIdx.x & 31, wid = threadIdx.x >> 5;
  if (lane == 0) ws[wid] = v;
  __syncthreads();
  if (wid == 0) {
    v = (lane < (blockDim.x >> 5)) ? ws[lane] : 0;
    v = warpSum(v);
    if (lane == 0) atomicAdd(out, v);            // out must be
     zeroed first
  }
}
// launch: gridDim ~ #SM * (resident blocks/SM), e.g. 4*#SM,
     blockDim 256
```

Traffic = n reads ≈ ขั้นต่ำ; work = $O(n)$; อิง Brent: p = thread ที่ resident, แต่ละตัวทำ n/p แบบ sequential.

10. Scan บน CUDA

Hillis–Steele ใน 1 block (double buffer)

```
__global__ void scanHS(const float *in, float *out, int n) { // n
     == blockDim.x
  extern __shared__ float tmp[];                          // 2*n
     floats
  int t = threadIdx.x, pout = 0, pin = 1;
  tmp[t] = in[t]; __syncthreads();
  for (int off = 1; off < n; off <<= 1) {
    pout = 1 - pout; pin = 1 - pout;
    tmp[pout*n+t] = (t >= off) ? tmp[pin*n+t] + tmp[pin*n+t-off] :
     tmp[pin*n+t];
    __syncthreads();
  }
  out[t] = tmp[pout*n + t];
}
```

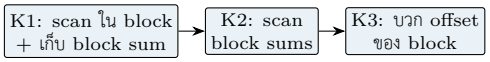
Bug คลาส สิก: in-place `tmp[t] += tmp[t- off]` → **race**: thread อื่น อาจ อ่าน ค่าที่ เพิ่ง ถูก เขียน ใน step เดียวกัน. แก้: double buffer หรือ `v = tmp[t-off]; __syncthreads(); tmp[t] += v; __syncthreads();`

Blelloch ใน 1 block ($n = 2\times$blockDim)

```
__global__ void scanBlelloch(const float *in, float *out, int n) {
  extern __shared__ float tmp[];
  int t = threadIdx.x, off = 1;
  tmp[2*t] = in[2*t]; tmp[2*t+1] = in[2*t+1];
  for (int d = n >> 1; d > 0; d >>= 1) {          // up-sweep
    __syncthreads();
    if (t < d) { int ai = off*(2*t+1)-1, bi = off*(2*t+2)-1; tmp[bi
     ] += tmp[ai]; }
    off <<= 1;
  }
  if (t == 0) tmp[n-1] = 0;                        // identity at
     root
  for (int d = 1; d < n; d <<= 1) {               // down-sweep
    off >>= 1; __syncthreads();
    if (t < d) { int ai = off*(2*t+1)-1, bi = off*(2*t+2)-1;
      float x = tmp[ai]; tmp[ai] = tmp[bi]; tmp[bi] += x; }
  }
  __syncthreads();
  out[2*t] = tmp[2*t]; out[2*t+1] = tmp[2*t+1];
}
```

Fine-tune: index stride 2^d → **bank conflict** สูงขึ้นทุกระดับ → **padding** `idx + (idx >> 5)` (เว้น 1 ช่องทุก 32). Warp-level scan ด้วย `__shfl_up_sync` (HS ใน warp, ไม่ต้อง sync).

Scan ขนาดใหญ่ (หลาย block)



- **Scan-then-propagate** (ข้างบน): traffic ≈ K1 อ่าน n เขียน n + K3 อ่าน n เขียน n = **4n**
- **Reduce-then-scan**: K1 reduce ต่อ block (อ่าน n) → K2 scan sums → K3 scan ใหม่พร้อม offset (อ่าน n เขียน n) = **3n**
- **Single-pass decoupled look-back** (CUB): block รอ/มองย้อน prefix ของ block ก่อนหน้า → **2n** (ขั้นต่ำ)
- block sums มากเกิน 1 block → ทำ recursive (เช่น $n = 2^{24}$, 1024 elem/block → 16384 sums → 16 → 1: K1,K1,K1,K3,K3 = 5 launches)
- ทั้งหมดยัง $W = O(n)$; scan เป็น memory-bound → วัดด้วย GB/s

11. Histogram บน CUDA

```
__global__ void histGlobal(const unsigned char *in, unsigned *bins,
      size_t n) {
  for (size_t i = blockIdx.x * blockDim.x + threadIdx.x; i < n;
       i += (size_t)blockDim.x * gridDim.x)
    atomicAdd(&bins[in[i]], 1u);          // n global atomics,
     contention!
}
#define NB 128
__global__ void histPrivate(const unsigned char *in, unsigned *bins
     , size_t n) {
  __shared__ unsigned sh[NB];
  for (int b = threadIdx.x; b < NB; b += blockDim.x) sh[b] = 0;
  __syncthreads();
  for (size_t i = blockIdx.x * blockDim.x + threadIdx.x; i < n;
       i += (size_t)blockDim.x * gridDim.x)
    atomicAdd(&sh[in[i] & 0x7F], 1u);   // fast shared atomic
  __syncthreads();
  for (int b = threadIdx.x; b < NB; b += blockDim.x)
    if (sh[b]) atomicAdd(&bins[b], sh[b]);  // <= NB global atomics
     /block
}
```

Global atomics: $n \to \le$ #blocks $\times NB$ (เช่น 120 blocks × 128 = 15,360). ปัญหาของ naive: latency สูง + **contention** (ข้อมูลกระจุก เช่น ตัวอักษรอังกฤษ/ภาพสีเดียว) → serialize.

Practice 7.1: 4096 bins, saturate ที่ 127

- private hist 4096 × 4 B = 16 KB/block — ใส่ smem ได้ แต่ **จำกัด occupancy** (เช่น 64 KB/SM → 4 blk) ทางเลือก: ใช้ counter 8/16-bit ใน smem (`uchar` 4 KB) เพราะต้องการแค่ ≤ 127
- **กับดัก overflow**: n ได้ถึง 2^{32}; ถ้าทุกตัวค่าเดียวกัน counter 32-bit วนกลับเป็น 0 ก่อน clamp ⇒ ต้อง **saturate ระหว่างทาง**: `if (sh[v] < 127) atomicAdd(&sh[v],1)` (หรือ `atomicMin` หลังบวก) และใช้ `size_t` เป็น index
- merge: `atomicAdd` แล้ว clamp ท้าย สุด `bins[b] = min(bins[b],127)` (หรือ merge แบบ saturating)

Fine-tune histogram

- **Interleaved/ coalesced partitioning** (grid-stride) ดี กว่า ให้ thread อ่าน chunk ต่อเนื่องของตัวเอง
- **Aggregation**: ค่าซ้ำติดกัน → สะสมใน register แล้ว atomic ครั้งเดียว เมื่อค่าเปลี่ยน (ลด contention)
- **Per-warp sub-histogram** (ลด contention ใน block; แลก กับ smem) / **replication** หลายสำเนา
- **Coarsening**: block น้อยลง → merge น้อยลง; อ่านแบบ `uchar4`/`int4` (vectorized)

- bin เยอะเกิน smem → sort-based (radix sort + run-length) หรือ privatize เป็นช่วง ๆ

12. Fine-tune principles: measurable / throughput / bandwidth

ตัวชี้วัด (ต้องวัดได้)

- **Time**: `cudaEventRecord/ElapsedTime`; warm-up, เฉลี่ยหลายรอบ, ระบุรวม/ไม่รวม memcpy
- **Effective bandwidth** $BW_{\text{eff}} = \dfrac{B_{\text{read}} + B_{\text{write}}}{t}$ → % of peak
- **Throughput**: elements/s, GFLOP/s เทียบ peak compute
- **Arithmetic intensity** AI = FLOP/byte; **Roofline**: perf ≤ min(peak FLOPs, $AI \times BW_{peak}$); ridge point = peak FLOPs/BW_{peak} (เช่น 19.5 TF / 1.55 TB/s ≈ 12.6). $AI <$ ridge → **memory-bound**
- **Occupancy**, achieved bandwidth, divergence (branch efficiency), bank conflicts, atomic throughput: Nsight Compute (`dram__throughput`, `sm__warps_active`)
- **Little's law**: bytes in flight = latency × BW (เช่น 500 ns × 900 GB/s = 450 KB) → ต้องมี warp/ILP พอ

ตัวอย่างคำนวณ

- Reduce 2^{24} float ใน 0.5 ms: 67.1 MB / 0.5 ms = **134 GB/s** = 14.9% ของ 900 GB/s
- Reduce 0.085 ms: **790 GB/s** = 87.7% (ใกล้ peak แล้ว เลิก tune)
- Scan 2^{24} float $4n$ traffic = 268 MB ใน 0.8 ms = **336 GB/s** = 37%
- SAXPY $y = ax + y$: 2 FLOP / 12 B ⇒ $AI = 0.17$ → memory-bound; reduce: 1 FLOP/4 B = 0.25
- Matmul tiled ($T \times T$ tile): AI $\propto T$ → compute-bound ได้

Maximize throughput

- **Work-efficient ก่อน**: Blelloch ≻ HS เมื่อ n ใหญ่; coarsening ให้ work รวม ≈ sequential
- **Latency hiding**: occupancy พอ + **ILP** (โหลดหลายค่าก่อนใช้, unroll)
- **ลด divergence**: ให้ active thread ติดกัน (sequential addressing) → ทั้ง warp idle พร้อมกัน
- **ลด sync/atomic contention**: warp shuffle, privatization, ลดจำนวน launch, aggregation
- **ลด instruction overhead**: unroll, template block size, หลีก `%` `/` (ใช้ bit op)
- **Load balance**: grid-stride ทำให้ทุก thread ได้งานเท่ากัน; ไม่ให้ block ท้ายว่าง

Utilize bandwidth

- **Coalesced**: thread ติดกันเข้าถึง address ติดกัน; SoA แทน AoS; aligned
- **Reuse ใน smem/register**: โหลดจาก global ครั้งเดียว (tiling)
- **ไม่มี bank conflict**: sequential addressing, padding `+1` / `idx>>5`
- **ลด traffic**: kernel fusion (map+reduce ใน kernel เดียว), ไม่เขียนผลกลางลง global, ใช้ type เล็กลง (`uchar/half`), single-pass scan ($4n \to 2n$)
- **Vectorized load** `float4`: ลดจำนวน memory instruction 4 เท่า
- **ลด host↔device**: PCIe ≪ DRAM BW; ใช้ pinned memory, streams overlap

13. แม่แบบคำตอบข้อสอบ

(A) "แสดงว่า algorithm นี้ได้ optimal speedup บน PRAM"

1) เขียน T_p (แยกเฟส local n/p + combine $\log p$) 2) $C_p = pT_p$ 3) เทียบ C_p กับ T^* 4) หาเงื่อนไข p ที่ทำให้ $C_p = \Theta(T^*)$ เช่น $p = O(n/\log n)$ 5) สรุป $S_p = \Theta(p)$ ในช่วงนั้น; ระบุ model (EREW)

(B) "ออกแบบ WS algorithm + pseudocode + วิเคราะห์"

1) ระบุ input/output, operator ($\oplus$, identity, associative?) 2) decompose: map / reduce / scan / histogram 3) pseudocode: `for all ... in parallel` เฉพาะส่วนขนาน, loop ระดับเป็น `for` ปกติ 4) วิเคราะห์ W, S ของแต่ละเฟสแล้วรวม (ต่อกัน=บวก) 5) เทียบ W กับ $T^* \Rightarrow$ work-optimal? 6) S ถึง lower bound $\Omega(\log n)$? 7) Brent: $T_p = O(W/p + S)$, optimal speedup เมื่อ $p = O(T^*/S)$ 8) memory model ที่ต้องใช้ (EREW/CREW/CRCW) + ถ้าไม่ work-optimal เสนอ **cascading** แก้

(C) "แปลงเป็น CUDA"

1) map forall → thread/ grid-stride 2) แต่ละ step → `__syncthreads` / launch ใหม่ 3) เลือก memory: input global (coalesced) → smem/register ทำงานใน block → เขียนผลน้อยที่สุด 4) hierarchy warp→block→grid 5) ระวัง race (double buffer), bounds check `i<n`, overflow, `size_t` 6) launch config: blockDim 128–512 (ทวีคูณ 32), gridDim ≈ หลายเท่าของ #SM

(D) "เสนอ idea fine-tune" — ปัญหา → principle → วิธีแก้ → วัดผล

ปัญหา: วัดได้ 134 GB/s (15% peak) *principle*: kernel memory-bound (AI 0.25) → ต้อง utilize bandwidth
สาเหตุ: uncoalesced / thread ครึ่งหนึ่ง idle / $\log n$ launch / smem bank conflict / atomic contention
วิธีแก้: grid-stride (coalesced+coarsened), first-add-during-load, sequential addressing, warp shuffle, privatization, padding, fusion
วัดผล: เวลา (cudaEvent), GB/s เทียบ peak, Nsight: dram throughput, branch efficiency, bank conflicts, occupancy; **หยุดเมื่อใกล้ roofline** (เช่น > 80% peak BW)

สรุปตาราง Work/Step

Algorithm	W	S	Opt?	Model
Tree reduce	$O(n)$	$O(\log n)$	Y	EREW
Cascaded sum	$O(n)$	$O(\log n)$	Y	EREW
Hillis–Steele scan	$O(n \log n)$	$\log n$	N	EREW
Blelloch scan	$\approx 2n$	$2\log n$	Y	EREW
Recursive scan	$O(n)$	$O(\log n)$	Y	EREW
Histogram matrix	$O(nm)$	$O(\log n)$	N	EREW
Histogram cascaded	$O(n)$	$O(m+\log n)$	Y	EREW
Pack/ split/enumerate	$O(n)$	$O(\log n)$	Y	EREW
Radix sort (b bits)	$O(bn)$	$O(b\log n)$	Y*	EREW
Pointer jumping	$O(n \log n)$	$O(\log n)$	N	CREW
Max (n^2 proc)	$O(n^2)$	$O(1)$	N	Common CRCW

*เทียบกับ sequential radix sort

Pitfalls ที่มักโดนหักคะแนน

- ลืม `__syncthreads()` ระหว่าง step หรือใส่ไว้ใน `if` ที่บาง thread ไม่เข้า (deadlock/UB)
- คิดว่า `__syncthreads` sync ข้าม block ได้ (ไม่ได้!)
- in-place HS scan (race), ลืม bounds check, `int` overflow เมื่อ

$n \geq 2^{31}$

- ลืม zero `out` ก่อน `atomicAdd`; ลืม merge private histogram
- อ้าง "เร็วขึ้น" โดยไม่มีตัวเลข — ต้องมี metric (ms, GB/s, % peak)
- ปน exclusive/inclusive; Blelloch ให้ **exclusive**, HS ให้ **inclusive**
- บอกว่า "W ดีกว่า sequential" — เป็นไปไม่ได้ ($W \geq \Theta(T^*)$)

14. Warp-level primitives (โค้ดสั้นที่ใช้บ่อย)

```
__inline__ __device__ float warpScanIncl(float v) {   // Hillis-
     Steele in a warp
  int lane = threadIdx.x & 31;
  for (int off = 1; off < 32; off <<= 1) {
    float y = __shfl_up_sync(0xffffffff, v, off);       // read
     before write: no race
    if (lane >= off) v += y;
  }
  return v;                                             //
     exclusive = v - x
}
// warp-aggregated compaction: keep x[i] > mu   (all 32 lanes must
     reach ballot)
bool keep = (i < n) && (x[i] > mu);
unsigned m = __ballot_sync(0xffffffff, keep);           // 32 flags
      -> 1 word
int lane = threadIdx.x & 31;
int pos  = __popc(m & ((1u << lane) - 1));              //
     exclusive scan of flags
int base = 0;
if (lane == 0) base = atomicAdd(&outCount, __popc(m));  // 1 atomic
      per warp
base = __shfl_sync(0xffffffff, base, 0);                //
     broadcast
if (keep) out[base + pos] = x[i];                       // NOT
     stable across warps
```

ต้องการผลแบบ **stable** (คงลำดับ) → ใช้ scan ระดับ block + scan ของ block counts แทน atomic

15. ตัวอย่างข้อสอบพร้อมคำตอบ (ย่อ)

Q1 (PRAM) คูณ n จำนวน: sequential, parallel, optimal?

Seq: `r=1; for i: r*=a[i]` → $n-1$ ครั้ง, $T^* = \Theta(n)$ (ต้องอ่านครบทุกตัว $\Rightarrow \Omega(n)$ จึง optimal)
Par EREW, $p = n/\log n$: (1) แต่ละ proc คูณ block ของตัวเอง $\log n$ ตัว: $O(\log n)$ (2) tree คูณ p ค่า: $O(\log p) = O(\log n)$ $\Rightarrow T_p = O(\log n)$, $C_p = \frac{n}{\log n} \cdot \log n = O(n)$ **cost-optimal** $\Rightarrow S_p = \Theta(p)$; ไม่มี access ชนกัน → EREW; step $\Omega(\log n)$ เป็น minimum (fan-in 2)

Q2 (WS) Max ของ n ค่า: pseudocode, W, S, Brent

ใช้ reduce tree แทน $\oplus$ ด้วย max: $W = n + n/2 + \cdots = O(n)$, $S = \log n + O(1)$ (EREW) $\Rightarrow T_p \leq n/p + \log n + O(1)$, optimal speedup เมื่อ $p = O(n/\log n)$. บน Common CRCW: $O(1)$ step แต่ $W = O(n^2)$ (ทุกตัวที่เขียน m_i เขียนค่า 0 เหมือนกัน จึงถูกกติกา Common)

Q3 (WS) Merge sort: $W(n) = 2W(n/2) + O(n)$, $S(n) = S(n/2) + O(\log n)$

$W = O(n \log n) = \Theta(T^*)$ **work-optimal**; $S = O(\log n + \log \frac{n}{2} + \ldots) = O(\log^2 n)$; optimal speedup ถึง $p = O(T^*/S) = O(n \log n/\log^2 n) = O(n/\log n)$

Q4 (WS) Matrix–vector $y = Ax$

forall i, j: $P[i][j] \leftarrow A[i][j]\,x[j]$; **forall** i: $y[i] \leftarrow$ REDUCE$(P[i][1..n], +)$
$W = O(n^2) = \Theta(T^*)$ **optimal**; $S = 1 + O(\log n)$ (reduce ทุกแถวขนานกัน: step = max ไม่ใช่ผลรวม). $x[j]$ ถูกอ่านพร้อมกัน n ตัว → **CREW** (ถ้าต้อง EREW: broadcast x ก่อน $O(\log n)$ step)

Q5 Scan trace $\langle 2,5,1,3,4,0,6,2\rangle$

incl $= \langle 2,7,8,11,15,15,21,23\rangle$; excl $= \langle 0,2,7,8,11,15,15,21\rangle$
HS: off1 2, 7, 6, 4, 7, 4, 6, 8 / off2 2, 7, 8, 11, 13, 8, 13, 12 / off4 2, 7, 8, 11, 15, 15, 21, 23
Blelloch: up0 2, 7, 1, 4, 4, 4, 6, 8 / up1 2, 7, 1, 11, 4, 4, 6, 12 / up2 $\ldots$, 23 / root←0 / dn2 2, 7, 1, 0, 4, 4, 6, 11 / dn1 2, 0, 1, 7, 4, 11, 6, 15 / dn0 0, 2, 7, 8, 11, 15, 15, 21
#ops: seq 7; HS $7+6+4 = 17$ $(= n\log n - (n-1))$; Blelloch $7+7 = 14$ $(= 2(n-1))$

Q6 Compaction: เก็บ เฉพาะ $A_i \geq 0$ จาก $\langle 3,-1,4,-2,-7,9,2,-5\rangle$

map $F = \langle 1,0,1,0,0,1,1,0\rangle$ → excl scan $X = \langle 0,1,1,2,2,2,3,4\rangle$ (total 4) → scatter $R[X_i] \leftarrow A_i$ ถ้า $F_i \Rightarrow \langle 3,4,9,2\rangle$; $W = O(n)$, $S = O(\log n)$

Q7 วงเล็บสมดุล

`(()(()))`: $D = 1,2,1,2,3,2,1,0$; min$= 0$, last$= 0 \Rightarrow$ **balanced**
`())(()`: $D = 1,0,-1,0,1,0$; min$= -1 \Rightarrow$ **ไม่ balanced** (แม้ผลรวม $= 0$)

Q8 Linear recurrence $x_i = a_i x_{i-1} + b_i$, $x_0 = 1$, $(a,b) = (2,1),(3,0),(1,4)$

$f_2(f_1(x)) = a_2(a_1x+b_1)+b_2 \Rightarrow (a_1,b_1)\otimes(a_2,b_2) = (a_1a_2,\ a_2b_1+b_2)$, $I = (1,0)$
Associative: ทั้งสองข้างได้ $(a_1a_2a_3,\ a_2a_3b_1 + a_3b_2 + b_3)$; **ไม่ commutative**: $(2,1)\otimes(3,0) = (6,3) \neq (3,0)\otimes(2,1) = (6,1)$
incl scan: $P_1 = (2,1)$, $P_2 = (6,3)$, $P_3 = (6,7)$; $x_i = A_i x_0 + B_i \Rightarrow 3, 9, 13$ (ตรวจ: 3, 9, $9+4 = 13$ ✓)

Q9 RLE ของ $\langle a,a,b,b,b,c,a,a\rangle$

head $f = \langle 1,0,1,0,0,1,1,0\rangle$ → excl scan $\langle 0,1,1,2,2,2,3,4\rangle$ (4 runs) → scatter $V = \langle a,b,c,a\rangle$, $Pos = \langle 0,2,5,6\rangle$, $Pos[4] = n = 8$ → $L = \langle 2,3,1,2\rangle$

Q10 Median ของคะแนน 0–100 / Mean–variance

Median: histogram ($m = 101$) → incl scan $C[s]$ → map s ถ้า $C[s] \geq \lceil n/2\rceil$ else ∞ → reduce(min). $W = O(n+m)$ (ดีกว่า sort $n\log n$), $S = O(\log n)$
Mean/var: map $(1, x_i, x_i^2)$ → reduce ด้วยการบวกทีละ component (associative, $I = (0,0,0)$); $\mu = s/n$, $\sigma^2 = q/n - \mu^2$ (**cancellation** ถ้า $\mu \gg \sigma$ → ใช้ Welford/Chan (n, μ, M_2))

Q11 (CUDA) วิเคราะห์ reduce0: `if (tid % (2*s) == 0) s[tid] += s[tid+s];`

$s = 1$, 256 thr: active 128 ตัวแต่กระจายทั้ง **8 warps** (แต่ละ warp ใช้ครึ่งเดียว)
ปัญหา: (1) warp divergence (2) `%` ช้า (3) ครึ่งหนึ่งของ thread idle ตั้งแต่ต้น (4) sync ทุก iteration แม้เหลือ warp เดียว (5) ถ้าแก้เป็น `idx=2*s*tid` จะเกิด bank conflict
แก้: sequential addressing + first-add-during-load + warp shuffle ช่วง 32 ตัวสุดท้าย + grid-stride; วัด: ms, GB/s vs peak, Nsight branch efficiency, bank conflicts

Q12 (CUDA) Hillis–Steele in-place มี bug อะไร

`t[tid] += t[tid-off]` อ่าน-เขียน array เดียวกันใน step เดียว → thread อาจอ่านค่าที่เพิ่งถูกอัปเดต (warp ต่างกันรันไม่พร้อมกัน) → ผลผิดแบบไม่แน่นอน, ผิด semantics ของ forall. แก้: อ่านลง register → `__syncthreads()` → เขียน → `__syncthreads()` หรือ double buffer

Q13 (CUDA) Scan $n = 2^{22}$, 1024 elem/block

4096 blocks → K1 scan+block sums (4096) → K2 scan sums ด้วย 4 blocks (ได้ 4 sums) → K3 scan 4 ค่า → K4 บวก offset เข้า S_1 → K5 บวก offset เข้าทุก element = **5 launches**, traffic ≈ $4n$. ลดเป็น $3n$ (reduce-then-scan) หรือ $2n$ (decoupled look-back), coarsening ลดจำนวน block sums

Q14 (CUDA) Histogram 128 bins, 10^8 chars, 120 blocks

Naive: global atomic 10^8 ครั้ง + contention (e, space พบบ่อย → serialize)
Privatized: shared atomic 10^8 (on-chip) + global ≤ $120 \times 128 = 15{,}360$ ครั้ง (∼6,500× น้อยลง)
เพิ่ม: aggregation, per-warp sub-hist, coarsening (blocks ≈ #SM×blk/SM), `uchar4` load; วัด: input GB/s, Nsight atomic throughput, stall reasons

Q15 (Design end-to-end) เลือกค่าที่มากกว่า mean ออกมา

High-level: reduce(+) → μ → map $F_i = [x_i > \mu]$ → excl scan(F) → scatter; $W = O(n)$, $S = O(\log n)$ work-optimal
Low-level: K1 reduce (grid-stride+shuffle+atomic, อ่าน n); K2 คำนวณ flag ใน register + นับต่อ block (อ่าน n); K3 scan block counts + scan ใน block + scatter (อ่าน n เขียน k)
Fine-tune: **fusion** ไม่เขียน flag/scan array ลง global (recompute ถูกกว่า): traffic ≈ $3n+k$ แทน ≈ $7n$; warp `__ballot_sync+__popc` แทน smem scan; coarsening `float4`; รวม output ใน smem แล้วเขียน coalesced
วัด: $BW_{\text{eff}} = 4(3n+k)/t$ เทียบ peak, > 80% ถือว่าใกล้ขีดจำกัด

Q16 (CUDA) Occupancy / Coalescing / Bank (คำตอบเร็ว)

256 thr, 32 reg → 100%; 64 reg → 50% (reg); 24 KB smem/blk (64 KB/SM) → 2 blk = 25% (smem); 32 thr/blk → 50% (max blk)
`A[i]` 100%, `A[2i]` 50%, `A[32i]` 12.5%, `A[i+1]` 80%, AoS 12 B 33%
`s[t]` 0, `s[2t]` 2-way, `s[32t]` 32-way, `s[33t]` 0 (= padding)

16. Host code & การวัดผล (template)

```
float *d_in, *d_out;  size_t bytes = n * sizeof(float);
cudaMalloc(&d_in, bytes);  cudaMalloc(&d_out, sizeof(float));
cudaMemcpy(d_in, h_in, bytes, cudaMemcpyHostToDevice);
int T = 256, B = 4 * numSMs;                          // grid-stride
      launch
cudaMemset(d_out, 0, sizeof(float));                  // before
     atomicAdd
reduceFinal<<<B, T>>>(d_in, d_out, n);                // warm-up
cudaEvent_t e0, e1; cudaEventCreate(&e0); cudaEventCreate(&e1);
cudaEventRecord(e0);
for (int r = 0; r < R; r++) {                         // average R
     runs
  cudaMemset(d_out, 0, sizeof(float));
  reduceFinal<<<B, T>>>(d_in, d_out, n);
}
cudaEventRecord(e1); cudaEventSynchronize(e1);
float ms; cudaEventElapsedTime(&ms, e0, e1); ms /= R;
double GBs = (double)bytes / (ms * 1e-3) / 1e9;       // effective
     bandwidth
printf("%.3f ms, %.1f GB/s (%.1f%% of peak)\n", ms, GBs, 100 * GBs
     / peakGBs);
// peak BW = memClock(Hz) * 2 (DDR) * busWidth(bits) / 8
```

- ตรวจ error: `cudaGetLastError()` หลัง launch, `cudaDeviceSynchronize()` ก่อนอ่านผล
- ตรวจความถูกต้องเทียบ CPU (float: ใช้ relative tolerance เพราะลำดับการบวกต่างกัน)
- ระบุชัดว่าเวลารวม/ไม่รวม `cudaMemcpy` (PCIe ≈ 16–32 GB/s ≪ DRAM)

17. Tiled matrix multiply (data reuse / arithmetic intensity)

```
#define TILE 16
__global__ void matmulTiled(const float *A, const float *B, float *
    C, int N) {
  __shared__ float As[TILE][TILE], Bs[TILE][TILE];
  int ty = threadIdx.y, tx = threadIdx.x;
  int row = blockIdx.y * TILE + ty, col = blockIdx.x * TILE + tx;
  float acc = 0;
  for (int t = 0; t < (N + TILE - 1) / TILE; t++) {
    int ac = t * TILE + tx, br = t * TILE + ty;
    As[ty][tx] = (row < N && ac < N) ? A[row*N + ac] : 0;   //
     coalesced
    Bs[ty][tx] = (br < N && col < N) ? B[br*N + col] : 0;
    __syncthreads();                                        // tile
     loaded
    for (int k = 0; k < TILE; k++) acc += As[ty][k] * Bs[k][tx];
    __syncthreads();                                        // before
     next tile
  }
  if (row < N && col < N) C[row*N + col] = acc;
}
```

Naive: แต่ละ FMA (2 FLOP) อ่าน 2 float (8 B) จาก global ⇒ $AI = 0.25$ (memory-bound). Tiled: global load ลดลง TILE เท่า ⇒ $AI \approx$ TILE/4 ($16 \to 4$, $32 \to 8$) → เข้าใกล้ compute-bound. `Bs[k][tx]` ไม่มี bank conflict; `As[ty][k]` เป็น broadcast. WS: $W = O(n^3)$, $S = O(\log n)$ (forall i, j, k + reduce บน k).

18. คำศัพท์ (EN → ความหมาย)

Speedup / Efficiency	T^*/T_p / speedup ต่อ processor
Cost-optimal	$pT_p = \Theta(T^*)$
Work-optimal	จำนวน op รวม $= \Theta(T^*)$
Step / Span / Depth	ความยาว critical path
Parallelism	W/S = จำนวน proc สูงสุดที่ใช้ได้คุ้ม
Associative	$(a \oplus b) \oplus c = a \oplus (b \oplus c)$ — จำเป็นสำหรับ reduce/scan
Identity	$I \oplus a = a$ (0 ของ +, 1 ของ ×, $-\infty$ ของ max)
Privatization	แยกสำเนาส่วนตัว แล้ว merge
Thread coarsening	1 thread ทำหลาย element (= cascading)
Coalescing	warp อ่าน address ติดกันใน transaction น้อยที่สุด
Bank conflict	thread ใน warp ชน bank เดียวกันใน smem → serialize
Warp divergence	thread ใน warp เดียวไปคนละ branch
Occupancy	active warps / max warps ต่อ SM
Latency hiding	สลับ warp ระหว่างรอ memory
Memory/compute-bound	ถูกจำกัดด้วย BW / FLOPs (เทียบ AI กับ ridge)
Kernel fusion	รวมหลาย kernel เพื่อลด global traffic